Demo Guide

NetworkPeer Executive Demo Guide & Hardware Installation

Version: 1.3
Date: 2026-09-02
Demo Duration: 5 minutes
Audience: Leadership / Stakeholders

Quick Reference: Demo Flow (5 Minutes)

Time	Action	Platform	Talking Point
0:00	Open app, enter phone	iOS + Android	“Unified auth across platforms”
0:30	Receive SMS, enter OTP	Both	“Cognito Custom Auth - no passwords”
1:00	Create job (Client)	iOS	“Client posts job in 30 seconds”
1:30	Worker sees job, accepts	Android	“Real-time marketplace matching”
2:00	Worker arrives, captures evidence	Android	“Photo/video with metadata”
2:30	Client reviews evidence	iOS	“Secure presigned download”
3:00	Job completion, payment trigger	Both	“End-to-end workflow”
3:30	Push notifications demo	Both	“Cross-platform real-time”
4:00	Admin dashboard (web)	Web	“Platform oversight”
4:30	Architecture overview	Slides	“Scalable, secure, cloud-native”

1. iOS Physical Device Installation Guide

1.1 Prerequisites

- **Mac** with Xcode 15+ (App Store)
- **iPhone** running iOS 17+
- **Apple ID** (free developer account works)

- **Lightning/USB-C cable**
- NetworkPeer iOS project: `/Users/adityasharma/Desktop/NETWORKPEER/apps/ios`

1.2 Step-by-Step

Step 1: Open Project in Xcode

```
cd /Users/adityasharma/Desktop/NETWORKPEER/apps/ios
open NetworkPeer.xcodeproj
```

Step 2: Configure Signing

1. Select **NetworkPeer** project in navigator
2. Select **NetworkPeer** target
3. **Signing & Capabilities** tab:
   - ✅ **Automatically manage signing**
   - **Team**: Select your Apple ID (Personal Team)
   - **Bundle Identifier**: `com.networkpeer.app` (or your unique ID)
   - If error: Change bundle ID to something unique like `com.yourname.networkpeer`

Step 3: Connect iPhone

1. Connect iPhone via cable
2. Unlock iPhone, tap **Trust** on device
3. In Xcode toolbar, select your iPhone from device dropdown (top left)

Step 4: Build & Run

1. Press **⌘R** (Run) or click **Play** button
2. First build takes 2-3 minutes (downloads dependencies)
3. App installs on iPhone

Step 5: Trust Developer Certificate (On iPhone)

1. **Settings → General → VPN & Device Management**
2. Tap your **Apple ID** under “Developer App”
3. Tap **Trust** → Confirm
4. Return to home screen, launch **NetworkPeer**

1.3 Troubleshooting

Issue	Solution
“No signing certificate”	Xcode → Settings → Accounts → + → Apple ID → Manage Certificates → + → iOS Development
“Device not found”	Window → Devices and Simulators → Check “Show as run destination”
Build fails on Firebase	Clean build folder (⌘ ⇧ K), rebuild
“Could not launch”	Delete app from iPhone, rebuild

1.4 Production Build (TestFlight/App Store)

```
# In Xcode:
# 1. Product → Archive
# 2. Distribute App → TestFlight / App Store Connect
# 3. Follow upload wizard
```

---

2. Android Physical Device Installation Guide

2.1 Prerequisites

- **Android Studio** (latest) or **Command-line tools**
- **Android Phone** with USB Debugging enabled
- **USB Cable**
- NetworkPeer Android project: `/Users/adityasharma/Desktop/NETWORKPEER/apps/android`

2.2 Enable USB Debugging (On Phone)

1. **Settings** → **About Phone** → Tap **Build Number** 7 times
2. **Settings** → **System** → **Developer Options** → ✅ **USB Debugging**
3. Connect via USB → Allow USB debugging on phone prompt

2.3 Option A: Android Studio (Recommended)

```
# 1. Open Android Studio
# 2. File → Open → /Users/adityasharma/Desktop/NETWORKPEER/apps/android
# 3. Wait for Gradle sync (2-3 min first time)
# 4. Connect phone via USB
# 5. Select device in toolbar dropdown
# 6. Click ▶️ Run (or ⌘R / Ctrl+R)
```

2.4 Option B: Command Line

```
cd /Users/adityasharma/Desktop/NETWORKPEER/apps/android

# Set environment (add to ~/.zshrc or ~/.bashrc for persistence)
export JAVA_HOME="/opt/homebrew/Cellar/openjdk@17/17.0.20.1/libexec/
        openjdk.jdk/Contents/Home"
export ANDROID_HOME="/opt/homebrew/share/android-commandlinetools"
export PATH="$ANDROID_HOME/cmdline-tools/latest/bin:$ANDROID_HOME/platform-
        tools:$PATH"

# Verify device connected
adb devices
# Should show: <serial>    device

# Build & install debug APK
./gradlew installDevelopmentDebug

# Or build APK only
./gradlew assembleDevelopmentDebug
# Output: app/build/outputs/apk/development/debug/app-development-debug.apk

# Install manually
adb install -r app/build/outputs/apk/development/debug/app-development-
        debug.apk
```

2.5 Verify Installation

- App appears in app drawer as **NetworkPeer**
- Launch → Should show phone entry screen

2.6 Production Build (Play Store)

```
# Generate signed bundle
./gradlew bundleProductionRelease

# Output: app/build/outputs/bundle/productionRelease/app-production-release.aab
# Upload to Play Console
```

---

3. Demo Environment Configuration

3.1 Backend URL Configuration

iOS (NetworkPeerAPI.swift)

```
// In Sources/NetworkPeerCore/NetworkPeerAPI.swift
// Change baseURL for demo:
private let baseURL = URL(string: "https://api.demo.networkpeer.com")!  //
        Production
// OR for local testing:
private let baseURL = URL(string: "http://192.168.1.xxx:3000")!  // Your Mac's
        LAN IP
```

Android (NetworkPeerApi.kt)

```
// In app/src/main/java/.../network/NetworkPeerApi.kt
// Change BASE_URL for demo:
private const val BASE_URL = "https://api.demo.networkpeer.com/"  // Production
// OR for local testing:
private const val BASE_URL = "http://192.168.1.xxx:3000/"  // Your Mac's LAN IP
```

3.2 Local Backend for Demo (Optional)

```
# On your Mac - run local backend
cd /Users/adityasharma/Desktop/NETWORKPEER/NetworkPeer-main

# Start with Docker (includes Postgres + Redis)
docker-compose -f docker-compose.prod.yml up -d

# Or run locally (requires local Postgres/Redis)
npm run dev

# Backend runs on http://localhost:3000
# Use your Mac's LAN IP for mobile devices: http://192.168.x.x:3000
```

3.3 Cognito Configuration for Demo

- **User Pool**: Use production or staging Cognito
- **Client ID**: Must match iOS Bundle ID / Android Package Name
- **Callback URLs**: Not needed for native (custom auth)
- **SMS**: Ensure SNS production access for real OTP delivery

---

4. Demo Script (5 Minutes) - Detailed

Minute 0:00-0:30 | Authentication

Action: Open both apps side-by-side **Say:** *“NetworkPeer uses phone-based authentication - no passwords, no social login friction. We use AWS Cognito Custom Auth with SMS OTP.”*

Steps: 1. Tap “Continue with Phone” on both devices 2. Enter same phone number (or two numbers if available) 3. Select role: **Client** (iOS), **Worker** (Android) 4. Tap “Send Code”

Expected: SMS arrives within 5-10 seconds on both devices

Minute 0:30-1:00 | OTP Verification

Action: Enter 6-digit code on both devices **Say:** *“OTP is verified server-side via Lambda challenge-response. Tokens are short-lived (1hr) with secure refresh rotation.”*

Steps: 1. Enter OTP digits (auto-advance fields) 2. Tap “Verify” 3. Both apps land on home screen

Expected: Successful login, role-based UI (Client sees “Post Job”, Worker sees “Available Jobs”)

Minute 1:00-1:30 | Create Job (Client - iOS)

Action: Create a job on iOS **Say:** *“Clients post jobs with location, budget, category. Geospatial indexing enables nearby worker discovery.”*

Steps: 1. Tap **+** or “Post Job” 2. Fill: Title “Office Plumbing Repair”, Description, Category “Plumbing” 3. Set location (tap map or use current) 4. Budget: $500 5. Tap “Publish”

Expected: Job appears in list with “OPEN” status

Minute 1:30-2:00 | Accept Job (Worker - Android)

Action: Worker accepts job on Android **Say:** *“Workers see nearby jobs in real-time. One-tap acceptance with optimistic UI.”*

Steps: 1. Android app shows new job in list (auto-refresh or pull) 2. Tap job → “Accept Job” 3. Confirm

Expected: Job status changes to “ASSIGNED”, client gets push notification

Minute 2:00-2:30 | Capture Evidence (Worker - Android)

Action: Worker adds evidence **Say:** *“Evidence capture with metadata - GPS, timestamp, media type. Direct S3 upload via presigned URLs - no backend bandwidth.”*

Steps: 1. Tap job → “Add Evidence” 2. Take photo (or select from gallery) 3. Add optional notes 4. Tap “Upload”

Expected: Upload progress → Success → Evidence appears in job detail

Minute 2:30-3:00 | Review Evidence (Client - iOS)

Action: Client reviews evidence **Say:** *“Clients review via secure, expiring download links. No direct S3 access - all mediated by API.”*

Steps: 1. iOS: Tap job → “Review Evidence” 2. Tap evidence item → Fullscreen viewer 3. Swipe through multiple items

Expected: High-res images load via CloudFront CDN

Minute 3:00-3:30 | Complete Job & Notifications

Action: Complete job on either device **Say:** *“Completion triggers payment workflow and notifications across all platforms.”*

Steps: 1. Worker: “Mark Complete” 2. Client: Receives push notification 3. Both: See “COMPLETED” status

Minute 3:30-4:00 | Cross-Platform Push

Action: Send test push from backend (or trigger via action) **Say:** *“Unified push abstraction - APNs for iOS, FCM for Android, Web Push for browser. Single API.”*

Steps: 1. Use admin endpoint or background worker to send test push 2. Both devices show notification 3. Tap notification → Deep links to job

Minute 4:00-4:30 | Admin Dashboard (Web)

Action: Open https://app.demo.networkpeer.com/admin **Say:** *“Real-time platform observability - jobs, users, revenue, evidence volume.”*

Show: - Live job count - Active users - Evidence uploads/minute - Error rate dashboard

Minute 4:30-5:00 | Architecture & Scalability

Action: Show architecture slide **Say:** *“Cloud-native on AWS: Fargate auto-scales, RDS Multi-AZ, ElastiCache, Cognito handles auth at millions of users. Terraform-managed, zero-downtime deployments.”*

---

5. Contingency Plans (If Things Go Wrong)

Failure Scenario	Backup Plan
No SMS delivery	Use pre-registered test numbers with known OTPs; or demo with “bypass” mode
Backend down	Run local backend on Mac (docker-compose)
iOS won’t build	Use iOS Simulator on Mac (⌘R in Xcode with simulator selected)
Android won’t install	Use Android Emulator in Android Studio
No internet	Pre-recorded video walkthrough (have ready on phone)
Push not working	Show notification center screenshot; explain architecture
Evidence upload slow	Use small test image; explain presigned URL direct-to-S3

Emergency Demo Mode (Local Only)

```
# 1. Start local stack
cd /Users/adityasharma/Desktop/NETWORKPEER/NetworkPeer-main
docker-compose -f docker-compose.prod.yml up -d

# 2. Update mobile apps to point to Mac's LAN IP
# Find IP: ifconfig | grep "inet " | grep -v 127.0.0.1

# 3. Run both apps on simulators/emulators
# iOS: Xcode → Simulator (iPhone 15 Pro)
# Android: Android Studio → Emulator (Pixel 8 API 34)

# 4. Demo entirely offline-capable
```

6. Demo Day Checklist

Night Before

- [ ] Charge both phones to 100%
- [ ] Verify cables work (test data transfer)
- [ ] Test full flow once end-to-end
- [ ] Record 2-min backup video of full flow
- [ ] Prepare slide deck with architecture diagram
- [ ] Print this guide (or have on iPad)

Morning Of

- [ ] Rebuild both apps (clean build)
- [ ] Test OTP delivery to your numbers
- [ ] Verify backend health (curl /health)
- [ ] Check push notifications work
- [ ] Verify admin dashboard loads
- [ ] Pack: Mac, both phones, cables, portable charger, HDMI adapter

At Venue

- [ ] Connect to WiFi / hotspot
- [ ] Test network connectivity to backend
- [ ]

- [ ] Launch apps, verify logged in
- [ ] Have backup video queued
- [ ] Relax - you know this system inside out

---

7. Key Talking Points (If Asked)

Question	Answer
“How does this scale?”	Fargate auto-scales 1→100 tasks; RDS read replicas; Cognito handles millions
“Is it secure?”	Cognito managed auth, TLS everywhere, encrypted storage, least-privilege IAM, no passwords stored
“What about offline?”	Optimistic UI, local queue, sync on reconnect (evidence, job updates)
“Platform fees?”	Configurable per-marketplace; demo shows $0 platform fee
“Integration?”	REST + WebSocket APIs; webhook support for ERP/accounting
“Compliance?”	SOC2-ready infrastructure; data residency via region selection; audit logs

---

8. Post-Demo Follow-Up

1. **Share repo access**: `https://github.com/rudraaxl/NetworkPeer`
2. **Send architecture doc**: `staging_build/architecture/TRACEABILITY_MATRIX.md`
3. **Send deployment runbook**: `staging_build/deployment/DEPLOYMENT_RUNBOOK.md`
4. **Schedule deep-dive** with engineering leads
5. **Collect feedback** for sprint planning

Good luck! The system is solid - demo with confidence.